

MÉMOIRE DE PROJET

SYSTÈME DE GESTION DE CRÈCHE

Application Web Complète

Développé avec Flask et MongoDB

Réalisé par : [Votre Nom]

Sous la direction de : [Nom du Superviseur]

Date : Décembre 2023

TABLE DES MATIÈRES

INTRODUCTION	1
CHAPITRE 1 : ANALYSE DES BESOINS	2
1.1 Présentation du projet	2
1.2 Objectifs	3
1.3 Acteurs et rôles	3
1.4 Fonctionnalités principales	4
CHAPITRE 2 : CONCEPTION DU SYSTÈME	6
2.1 Architecture générale	6
2.2 Modèle de données (ERD)	7
2.3 Cas d'utilisation	8
2.4 Diagrammes UML	9
CHAPITRE 3 : RÉALISATION	10
3.1 Technologies utilisées	10
3.2 Structure du projet	11
3.3 Implémentation des contrôleurs	12
3.4 Interface utilisateur	13
CHAPITRE 4 : TESTS ET VALIDATION	14
4.1 Stratégie de test	14
4.2 Tests unitaires	14
4.3 Tests d'intégration	15
CHAPITRE 5 : CONCLUSION	16
BIBLIOGRAPHIE	17
ANNEXES	18

INTRODUCTION

Dans le contexte actuel où la gestion des établissements éducatifs devient de plus en plus complexe, il est essentiel de disposer d'outils informatiques performants pour faciliter les tâches administratives et pédagogiques. Ce mémoire présente le développement d'un système de gestion de crèche complet, permettant de gérer efficacement les enfants, les parents, les éducateurs et les diététiciens. Le projet a été réalisé en utilisant les technologies web modernes : Flask comme framework backend, MongoDB comme base de données NoSQL, et Bootstrap pour l'interface utilisateur. L'application offre une gestion complète des présences, des activités, de la nutrition et des paiements. Ce document détaille l'ensemble du processus de développement, de l'analyse des besoins à la mise en production, en passant par la conception et l'implémentation.

CHAPITRE 1 : ANALYSE DES BESOINS

1.1 Présentation du projet

Le système de gestion de crèche est une application web complète conçue pour automatiser et simplifier la gestion quotidienne d'une crèche. Elle permet aux différents acteurs (administrateurs, éducateurs, diététiciens et parents) d'accéder à des fonctionnalités adaptées à leurs rôles respectifs.

1.2 Objectifs

Les objectifs principaux du projet sont : - Automatiser la gestion des présences des enfants - Faciliter la communication entre les différents acteurs - Gérer efficacement les plans nutritionnels - Simplifier les processus de paiement - Fournir des tableaux de bord personnalisés pour chaque rôle

1.3 Acteurs et rôles

Rôle	Responsabilités
Administrateur	Gestion complète du système, utilisateurs, configuration
Éducateur	Gestion des présences, activités, groupes d'enfants
Diététicien	Gestion de la nutrition, menus, plans alimentaires
Parent	Consultation des informations de leurs enfants, paiements

1.4 Fonctionnalités principales

Le système offre les fonctionnalités suivantes : - Gestion complète des utilisateurs et profils - Système de présence quotidien avec statistiques - Planification et gestion des activités - Gestion des groupes d'enfants - Création et gestion des repas et menus - Plans nutritionnels personnalisés - Système de communication (messages, plaintes) - Gestion des paiements avec intégration Chargily

CHAPITRE 2 : CONCEPTION DU SYSTÈME

2.1 Architecture générale

L'architecture du système suit le pattern MVC (Modèle-Vue-Contrôleur) avec Flask. L'application est structurée autour de blueprints pour une meilleure organisation. MongoDB est utilisé comme base de données NoSQL pour sa flexibilité et ses performances. L'architecture comprend : - Couche présentation : Templates Jinja2 avec Bootstrap - Couche contrôleur : Blueprints Flask pour chaque module - Couche modèle : Classes Python pour l'interaction avec MongoDB - Base de données : MongoDB avec collections pour chaque entité

2.2 Modèle de données (ERD)

Le modèle de données comprend les entités principales suivantes : - User : Gestion des comptes utilisateurs - Parent : Informations des parents - Enfant : Données des enfants - Educateur : Profils des éducateurs - Dietitian : Profils des diététiciens - Group : Groupes d'enfants - Presence : Enregistrements de présence - Meal_Record : Enregistrements des repas - Invoice : Factures et paiements Les relations entre ces entités sont gérées via des références MongoDB.

CHAPITRE 3 : RÉALISATION

3.1 Technologies utilisées

Technologie	Version	Utilisation
Flask	2.2.5	Framework web Python
MongoDB	4.4+	Base de données NoSQL
PyMongo	4.4.0	Driver MongoDB pour Python
Flask-Login	0.6.2	Gestion de l'authentification
Bootstrap	5.x	Framework CSS
Jinja2	-	Moteur de templates
ReportLab	3.6.13	Génération de PDF
Requests	2.31.0	Appels API externes

3.2 Structure du projet

L'organisation du code suit les bonnes pratiques Flask : gestion_creche_final/ ■■■■ app.py # Point d'entrée ■■■■ db.py # Configuration MongoDB ■■■■ controllers/ # Logique métier (Blueprints) ■■■■ models/ # Modèles de données ■■■■ templates/ # Templates Jinja2 ■■■■ static/ # Fichiers statiques ■■■■ utils/ # Utilitaires ■■■■ diagrams/ # Documentation UML

CHAPITRE 4 : TESTS ET VALIDATION

La validation du système comprend plusieurs niveaux de tests : - Tests manuels des fonctionnalités principales - Vérification de l'interface utilisateur - Tests d'intégration avec MongoDB - Validation des processus métier - Tests de sécurité de base L'application a été testée avec différents navigateurs et scénarios d'utilisation.

CHAPITRE 5 : CONCLUSION

Ce projet a permis de développer une application web complète pour la gestion d'une crèche, répondant aux besoins identifiés lors de l'analyse. L'utilisation de technologies modernes comme Flask et MongoDB a permis de créer un système robuste et évolutif. Les fonctionnalités implémentées couvrent l'ensemble des aspects de la gestion d'une crèche : administration, suivi pédagogique, nutrition et communication. L'architecture modulaire facilite la maintenance et l'évolution future du système. Ce travail démontre l'importance des outils informatiques dans la gestion des établissements éducatifs et ouvre des perspectives d'amélioration continue.

BIBLIOGRAPHIE

- [1] Documentation Flask. <https://flask.palletsprojects.com/>
- [2] Documentation MongoDB. <https://docs.mongodb.com/>
- [3] Documentation Bootstrap. <https://getbootstrap.com/>
- [4] Documentation ReportLab. <https://www.reportlab.com/docs/reportlab-userguide.pdf>
- [5] Python Documentation. <https://docs.python.org/3/>

ANNEXES

Annexe 1 : Code source principal (app.py) Annexe 2 : Modèles de données Annexe 3 : Diagrammes UML Annexe 4 : Captures d'écran de l'application Annexe 5 : Guide d'installation et de déploiement